E-Commerce Sales & Customer Analytics Project

A Complete SQL Portfolio & Interview Briefing Guide

PROJECT PARAMETERS:

SQL Dialect: PostgreSQL (Canonical) | SQLite (Validation Testing)

Dataset Scale: 2,000 Orders | 4,967 Line Items | 90 Registered Customers

Analytical Scope: 52 Business Questions Resolved

Key Analytical Models: RFM Segmentation, Basket Cohorts, Churn Tracking

Prepared by: aman

Role focus: Senior Data Analyst, SQL Developer & Analytics Lead

Date: August 2026

1. Executive Summary & Project Intent

This project was designed to transform a raw e-commerce database script into a professional-grade SQL analytics portfolio. Far from being a simple homework assignment, this implementation models the standards of a Senior Data Analyst in an enterprise setting. It establishes rigorous metric definitions, performs a schema and data quality audit, constructs reusable analytical views, and cross-reconciles all metrics to guarantee 100% mathematical integrity.

Key analytical highlights include the extraction of a 100% repeat purchase rate inside the buying cohort, the identification of a severe revenue category concentration risk (63.4% of total sales coming from Electronics), and a dynamic RFM customer segmentation model that groups active buyers into targetable marketing buckets.

2. Technical Stack & Architecture Rationale

Our choice of technologies was driven by industry standards, engineering efficiency, and the need for rigorous validation. Below is the rationale behind our architectural design:

PostgreSQL as the Canonical Environment

PostgreSQL is chosen as the canonical production dialect. It is the enterprise standard for relational databases, offering advanced support for date math, window partitioning, and standard CTE planning. In our DDL script (schema_postgresql.sql), we stripped out database-session parameters such as USE E_Commerce_Project which are incompatible with PostgreSQL, providing a clean schema loading DDL.

Python for the Automation Test Harness

A Python script (verify_results.py) was built to act as our validation framework. In an enterprise environment, analytics code must be validated before it is deployed. The Python test suite reads our canonical PostgreSQL views and queries, translates date and casting syntax into SQLite equivalents on-the-fly, and loads them into a test database to run automated semantic reconciliation checks.

SQLite for Portability and CI/CD Verification

We utilized an in-memory SQLite database as our validation test engine. Using SQLite allowed us to execute and verify all 52 query files and reusable views instantaneously in a sandboxed, zero-dependency local environment, guaranteeing execution correctness without requiring an active external server connection during testing.

3. Data Model, Grains, and Join Multiplication Risks

Understanding granularity is the single most critical task for an analyst. Working at the incorrect grain causes mathematical inflation, duplicate counts, and distorted metrics. Below is our schema grain log:

* **Customer Grain:**One row per customer registration. Core key is customer_id. Demographic attributes include city and gender.
* **Order Grain:**One row per checkout transaction. Core key is order_id. Contains customer_id, order_date, and order_status.
* **Product Grain:**One row per product SKU in the catalog. Core key is product_id. Contains category and unit_price.
* **Order-Item Grain:**One row per product line item within an order. Core key is order_item_id. References order_id and product_id.

The Risk of Join Multiplication

A common mistake made by junior analysts is joining the order table directly to the order-item table, and then performing order-level aggregations (like counting orders or averaging order totals). Because order_items is at a finer grain, this join repeats the order header attributes for every item in the order. If an order has 4 items, the order header repeats 4 times. A simple COUNT(order_id) would count that order 4 times, inflating transaction volume.

To mitigate this risk, we perform aggregations at the correct grain first - using CTEs or grouping by unique keys before conducting joins - or utilize DISTINCT parameters (such as COUNT(DISTINCT order_id)) to protect the denominator.

4. Data Quality & Schema Audit

Before conducting analysis, we performed a thorough database audit to identify anomalies. Below are the findings:

* **Duplicate Customer Names:**We found 5 duplicate names sharing different IDs (e.g. two different customers named 'Aditya Verma' with IDs 17 and 57). Grouping by customer name instead of customer ID will merge separate profiles and corrupt CRM targeting.
* **Never-Purchased customers:**Out of 90 registered profiles, 10 (11.11%) have never placed an order. They represent a marketing registration-to-activation gap.
* **Order Status Distribution:**Out of 2,000 orders placed, 1,649 are Delivered (82.45%), 240 are Cancelled (12.00%), and 111 are Pending (5.55%). Cancelled orders represent 5.21M (12.7% of gross bookings) in failed sales.
* **Price and Quantity Integrity:**We verified that 100% of order items refer to valid orders and valid products. There are zero orphans, negative values, or zero prices. Sales prices match product catalog prices in 100% of cases.

5. Business Metric Register & Assumptions

Ambiguous definitions lead to conflicting dashboards. We established a formal register for key metrics:

Recognized Revenue Definition

We define recognized revenue strictly as the sum of quantity * unit_price of orders with status = 'Delivered'. Delivered orders represent recognized revenue (32,280,641.00). Cancelled orders (5,213,749.00) are excluded because they represent failed bookings. Gross Booking Value (GBV) represents all orders placed (39,553,815.00), which we report separately as a sales pipeline health metric.

The Dynamic Temporal Anchor

For inactivity and recency, standard functions like CURRENT_DATE would make our queries permanently stale over time since the dataset stops on 2026-07-25. We derived a dynamic anchor using a CTE: SELECT MAX(order_date) FROM orders. This anchor (2026-07-25) is used as our baseline, and all time boundaries are calculated relative to it: 90-day inactivity cutoff is 2026-04-26, and 120-day churn risk cutoff is 2026-03-27.

Customer Cohorts & Taxonomy

Never-purchased customers are isolated from the churn cohort. A user who never bought represents an onboarding activation failure, whereas a user who stopped buying represents a retention failure. Churn rate is calculated strictly on the ordering cohort (80 active buyers) using a 90-day inactivity window. This gives a true churn rate of 10.00% (8 customers), while registered user inactivity stands at 20.00% (18 users, which includes the 10 who never purchased).

Historical Customer Lifetime Value (CLV)

Customer Lifetime Value (CLV) is calculated as the sum of recognized revenue per customer ID over the observed database period. This is a historical spend calculation. True predictive CLV is not supported due to missing behavioral data and margin data, which we explicitly call out in our limitations.

No Profitability Calculations

Because product cost data (COGS) is completely missing from the database, we assume product and category profitability cannot be calculated. Reusable product views focus strictly on unit volume, orders, realized revenue, ASP, and contribution.

6. Semantic Reconciliation & Validation Checks

Before exporting analytical insights, the database was subjected to our reconciliation suite (reconciliation_checks.sql), which cross-verifies summaries at different grains. All tests passed successfully:

* **Revenue Reconciliation:**Total sum of line items in order_items for delivered orders equals the sum of order-level revenues in our order summary: 32,280,641.00.
* **Product-to-Revenue:**The sum of product recognized revenues in the product summary equals total recognized revenue.
* **Category-to-Revenue:**The sum of category revenues matches recognized revenue exactly.
* **Order Status Consistency:**Delivered (1,649) + Cancelled (240) + Pending (111) equals total orders (2,000).
* **Ranking Integrity:**Confirmed that customers with higher recognized spends have ranks equal to or better than lower spending customers.
* **Top-N Containment:**The top 5 best sellers by volume are verified subsets of the top 10.
* **Contribution Percentages:**The sum of all product contribution percentages adds up to 100.0001% (within 0.05% rounding tolerance).
* **Pair Symmetry:**The co-occurrence matrix isolates undirected pairs (Product A < Product B), showing zero duplicate permutations.

7. Key Findings & Business Insights

Our analysis revealed several critical business trends and vulnerabilities:

* **Revenue Acceleration:**We generated 32.28M in recognized revenue, with a massive volume ramp from ~650k in Jan 2025 to over 4.15M in July 2026.
* **Portfolio Concentration Risk:**Electronics represents 63.43% of total revenue. The Smartphone X12 is our largest single SKU, contributing 22.37% (7.22M) of total sales. This represents extreme product concentration.
* **Geographic Strengths:**Mumbai is our top city by value, contributing 5.19M, followed by Pune and Delhi.
* **Funnel Vulnerability:**Our repeat purchase rate of the active cohort is 100%. Every buyer has purchased at least twice, with an average cycle of 15.87 days. However, customer acquisition is very slow (~3 new signups/month), meaning we are highly dependent on a stagnant customer base.
* **Affinity Pairings:**Market basket analysis shows that cross-category items frequently co-occur. Our top pair is Smartphone X12 & Skincare Kit (Electronics & Beauty, 13 co-occurrences).

8. Strategic Business Recommendations

Based on the analytical findings, the following actions are recommended to drive business value:

* **VIP Account Management:**Establish a VIP retention tier for the top 10 customers (spending > 900k each) who generate nearly a fifth of total sales. Offer direct support, priority delivery, and custom loyalty events.
* **Funnel Activation:**Deploy a welcome discount sequence (e.g. 15% discount) to activate the 10 cold customer accounts who registered but never ordered.
* **AOV Optimization:**Set up digital bundles and 'frequently bought together' triggers. Pack the Mechanical Keyboard with the Fiction Novel Pack as a 'Work From Home Bundle' to increase AOV.
* **Supply Chain Resilience:**Secure secondary electronics vendors. Our high reliance on the Smartphone X12 means any stockout will impact cash flow.
* **Category Diversification:**Diversify advertising spend into Clothing (+72.6% YoY growth) and Beauty (+56.9% YoY growth) to build secondary revenue streams.

9. Data Limitations (What the Data Cannot Tell Us)

A senior analyst must be honest about data limitations. The current dataset lacks critical attributes, preventing the calculation of several key metrics:

* **Profitability:**We cannot calculate margins or profitability. A high-revenue product like Smartphone X12 might have thin margins and be less profitable than a low-revenue book. We need a cost/COGS column in the products table.
* **Marketing ROI:**We cannot measure campaign efficiency, Customer Acquisition Cost (CAC), or marketing ROI. We need a marketing attribution table linking customer_id to acquisition sources.
* **Inventory Velocity:**We cannot measure stockout risks, replenishment cycles, or warehousing capacity. We need an inventory level and supplier replenishment log.
* **Shipping Performance:**We cannot measure shipping carrier efficiency or delivery delays. We need shipment tracking dates (shipped_date, delivered_date) and carrier costs.

10. Senior Analyst Interview Preparation Guide

Use this section to prepare for interviews. It outlines common questions a hiring manager might ask about this project, and the exact structured answers that demonstrate senior-level maturity.

Q1: Why did you choose PostgreSQL as the canonical SQL environment?

Response:
PostgreSQL is the industry standard for production relational databases, offering robust support for window functions, date formatting, and CTE optimization. The source file was labeled as PostgreSQL but contained MySQL-style CREATE DATABASE and USE syntax. I cleaned the script into a standard, executable DDL, separating database configuration from core schemas. For portability testing, I created a Python-based SQLite harness, translating dates and window logic on-the-fly, showing that I can manage cross-platform database migration.

Q2: How did you identify duplicate customer profiles, and why does it matter?

Response:
During my initial database audit, I checked for customer names associated with multiple customer IDs. I found 5 names (like Aditya Verma) that had separate IDs. Grouping by customer name instead of customer ID would merge distinct buyer profiles. This is a severe analytical risk; it would distort our CRM metrics and customer targeting. Throughout the project, I enforced customer_id as the primary grouping key to protect customer integrity.

Q3: How did you handle date-based analysis in a historical database?

Response:
A common junior mistake is using CURRENT_DATE for recency and inactivity. Since the dataset's transactions stop on 2026-07-25, using CURRENT_DATE today would show all customers as thousands of days inactive, returning empty tables. I centralized the date logic by deriving the analytical anchor dynamically: SELECT MAX(order_date) FROM orders. All calculations (90-day inactivity, 120-day churn risk, RFM recency) are anchored to this date. This ensures the scripts are fully reproducible and will not break when run in the future.

Q4: Why did you separate 'Never-Purchased' customers from the churn rate?

Response:
Never-purchased customers signed up but never ordered (10 customers). Churned customers purchased but stopped buying (8 customers). Combining them under churn rate misdiagnoses marketing acquisition problems as product retention problems. I isolated the cohorts, calculating a true buyer churn rate of 10.0% of the purchasing cohort, and a separate never-purchased activation gap of 11.11% of the registered base. This allows target teams to deploy separate campaigns: welcome discounts for activation, and re-engagement offers for churned users.

Q5: What is the risk of join multiplication, and how did you prevent it?

Response:
When joining orders with order_items, the order row repeats for every item in the order. If you run a simple SUM or AVG on order values after this join, the values will be inflated. I prevented this by either grouping by order_id first, using distinct parameters in aggregate functions, or deploying pre-aggregated views (like vw_order_summary) that consolidate items at the order grain before joining to customers. This keeps our order and customer counts mathematically sound.

Q6: How did you design your RFM scoring, and what did you discover?

Response:
I calculated Recency (days since last order), Frequency (count of delivered orders), and Monetary (LTV) for the active buying cohort. I assigned scores 1-5 using NTILE buckets, reversing the recency sort so that the most recent purchases received a 5. I then segmented the base. I discovered that nearly half of our buyers (47.5%) are 'Potential Loyalists' (moderate recency and frequency), representing a massive upsell opportunity. Meanwhile, we have 5 high-value customers flagged as 'At Risk' who haven't ordered in ~45 days, requiring immediate retention efforts.

Q7: What does the data not tell us, and what additions would you request?

Response:
The database lacks cost/COGS data, meaning we cannot calculate profitability or margin. High revenue does not mean high profit. We also lack marketing channels, preventing us from calculating CAC or marketing ROI. To expand the project, I would request a product cost table, marketing attribution logs (UTM parameters), and inventory logs.

Q8: How did you validate that your analytical views were correct?

Response:
I built a validation suite containing 8 semantic checks: reconciling order items to order summaries, verifying that category and product summaries sum exactly to total recognized revenue, validating status counts, checking ranking sequence consistency, and verifying pair symmetry in co-occurrences. I built a Python testing script to execute these automatically. Every single check returned PASSED, ensuring my reports match the database with precision.

Q9: What is the difference between Monthly Active Customers and true Cohort Retention?

Response:
Monthly Active Customers is a simple count of unique buyers who transacted in a month. True Cohort Retention groups customers by their acquisition/signup month (cohort) and tracks what percentage of that specific cohort returns to buy in Month 1, Month 2, etc. Presenting active user counts as retention is an analytical error. I clearly separated the active count trend from the cohort retention logic in my database and README.

--- END OF BRIEFING - READY FOR PRESENTATION ---